

Agent Zoo — Technical Implementation Report

Engineering reference for the agents in `opposition-agents-playing-mtg`

Abstract

This document is the engineering counterpart to the scientific writeup *Opposition Agents Playing Magic: The Gathering* (`paper/opposition_agents_mtg.tex`). Where the paper focuses on *objectives*, this report focuses on *exactly what the code does*: class signatures, control flow, configuration tables, failure modes, and the structured reasoning traces each agent emits. All cited paths are workspace-relative.

Contents

1	Theoretical background (1 page)	2
2	The contract every agent satisfies	3
3	The lineup	4
4	Glossary and symbols	4
5	Per-agent reference	4
5.1	RandomAgent	4
5.2	HeuristicAgent	5
5.3	KGHeuristicAgent	5
5.4	OllamaAgent (and LLMAgent alias)	5
5.5	ActiveInferenceAgent / KLControlActionPicker	6
5.6	WorldModelAgent	7
5.7	LLMFusionAgent	8
5.8	HumanAgent	9
6	Debugging recipes	9
6.1	Agent loops on pass priority	9
6.2	Agent never attacks	9
6.3	World-model agent regresses after checkpoint swap	9
6.4	LLM fusion hangs or times out	10
7	Reasoning trace contract	10
8	Test pointers	11
9	Theoretical foundations (extended)	11
9.1	POMDPs and partial observability	11
9.2	World models as latent imagination	11
9.3	Free energy, active inference, and optimal control	11
9.4	Knowledge-graph grounding	12

9.5	LLM-as-policy under legal-action constraints	12
9.6	Self-play promotion as fixed-point iteration	12
10	Ontology-backed decision provenance	12
11	Where to look next	13
12	References	13

Runtime status for the October 2026 review

The current native target is phase-rs 0.102.0, protocol 106 [5], from <https://github.com/phase-rs/phase>. The bridge sends engine-authored complete format configurations and retains incomplete outcomes for time, turn, action and transport limits. Four-player Commander currently means one Python-controlled seat and three native AI opponents with explicit decks, not four independent Python agents; terminal pods remain unqualified.

Full-session reconnect now retains the server-issued generation and player credential, submits the native `Reconnect` frame, and requires a matching `GameStarted` session, controlled seat and non-stale revision before reporting success. Credentials are not included in gameplay traces. A successful handshake alone does not restore a game.

The analytic KL model distinguishes cancellation from casting subdecisions. For an observed pending cast it forecasts one generic completion macro-step; repeated rollout subdecisions cannot repeatedly create board value, and these internal choices do not invent opponent attacks or advance the turn. This repairs the cancellation/mode tie. In two-player snapshots, simple fixed-damage player/object targets use native effect/target identities, including self-versus-opponent direction. Multiplayer snapshots and other effects retain coarse forecasts: the latent opponent-life minimum is not a specific player's life. This is not a general card-, mode- or target-exact transition model. Native interaction planning and matched objective budgets remain separate research gates.

Ontology construction is author-directed and LLM-assisted at the schema/tooling level; the rule-vocabulary generator performs deterministic parsing without runtime LLM calls. The companion ontology paper separates requirements, modelling, extraction, validation, revision and packaging, and discloses incomplete historical prompt/model provenance. The ontology vocabulary has been refreshed from the September 25 rules revision. Direct document provenance no longer depends on materialising the optional local rule-text graph. Corpus audits declare format and auxiliary-object exclusions and retain unknown-type denominators; resource validation results belong in the companion ontology paper rather than being treated as agent performance.

At runtime the local type/subtype vocabulary is an in-process cached index. Strategic graph access is optional Neo4j/Cypher, with partial caches in the KG-aware heuristic. An opt-in localhost read-only Fuseki snapshot is now implemented and query-verified, but not wired into agent decisions. Current n10s setup uses v2.0 Turtle, preserves compatible configuration and checks import counts; existing KG fallback behaviour and live writes still need qualification before live ontology grounding or successful writes are claimed. Docker availability is not required for the offline factual resource audits reported in the companion paper.

1 Theoretical background (1 page)

The framework treats Magic: The Gathering (MTG) as a partially-observable Markov game. Each player's perspective is a POMDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, T, \Omega, R, \gamma)$ with hidden state (opponent hands and library order). Concretely:

- $\mathcal{S}$: full `GameState` including all players' zones, the stack, mana pools, life totals, and book-keeping flags. Conservatively $|\mathcal{S}| > 10^{800}$.
- $\mathcal{O}$: seat-filtered snapshot emitted by phase-rs `StateUpdate`, then projected into our policy view by `src/integrations/phase_rs/adapter.py`.
- $\mathcal{A}$: legal action list from phase-rs wire messages, mapped into internal `Action` values by the same adapter, typically $|\mathcal{A}| \in [1, 60]$.
- T : phase-rs runtime transition system (Rust engine), deterministic given the random seed and chosen actions.
- R : terminal ± 1 on win/loss; agents may shape via Δlife , Δcards , Δtempo when training a value head.

Three families of policies. The agents implement:

1. *Reactive* (Random, Heuristic, KG-Heuristic, LLM): map $o_t \rightarrow a_t$ directly, no internal state.
2. *Model-based* (World-Model): learn $\hat{T}$ in latent space and pick $\arg \max_a \mathbb{E}_{\hat{T}}[V(s_{t+H})]$.
3. *Belief-driven* (KL-Control, LLM-Fusion): maintain a belief b_t over hidden state and minimise $G(\pi) = \text{KL}(Q(\eta|\pi) \| P_0(\eta)) + \mathbb{E}_Q[V(\eta)]$ closed-loop over that belief.

A self-contained derivation is in §9. See also the corresponding section in the companion paper.

The same formal stack also supports a collective-intelligence workflow: different agent families generate trajectories, those trajectories are mined into append-only KG evidence with provenance, and later agents use that shared graph memory as an additional prior. This separates immutable card facts from learned strategic evidence while allowing cumulative cross-agent learning across runs.

2 The contract every agent satisfies

Every agent class implements a single method:

```

1 class MTGAgent(Protocol):
2     name: str
3
4     async def decide_action(
5         self,
6         game_state: GameState,
7         legal_actions: list[Action],
8     ) -> Action: ...

```

Agents must *never* mutate `game_state`; only the engine is allowed to. An agent that returns an action not in `legal_actions` is treated as a forfeit by the phase-rs integration runner.

Determinism: any RNG owned by an agent must be a `random.Random` instance constructed from the seed passed by the runner. This guarantees (`seed`, `decklists`, `agent_types`) replays bit-exactly.

Agent	Module	Strategy
Random	<code>random_agent.py</code>	Uniform over legal actions.
Heuristic	<code>heuristic_agent.py</code>	Hand-tuned action priorities.
KG-Heuristic	<code>kg_heuristic_agent.py</code>	Heuristic + Neo4j combo lookup.
Ollama (LLM)	<code>ollama_agent.py</code>	Local LLM picks an action index.
KL-Control	<code>kl_control.py</code>	Belief-space KL control / path-integral MPC.
World-Model	<code>world_model_agent.py</code>	V+M+C latent rollouts.
LLM-Fusion	<code>llm_fusion_agent.py</code>	Weighted vote of all four signals (legacy).
Human	<code>human_agent.py</code>	Stdin prompt, used for debugging.

Symbol / term	Expanded form	Meaning in this repository
<i>POMDP</i>	Partially Observable Markov Decision Process	Formal model of each seat’s decision process under hidden information.
V, M, C	Encoder, Dynamics, Controller	World-model split: latent encoder, recurrent transition model, and policy/value heads.
<i>JEPA</i>	Joint Embedding Predictive Architecture	Predictor head trained in latent space without reconstruction.
<i>EFE</i> / $G(\pi)$	Expected Free Energy	Active-inference objective combining pragmatic and epistemic terms.
<i>KG</i>	Knowledge Graph	Neo4j-backed symbolic substrate over cards, combos, mechanics, and archetypes.
<i>APNAP</i>	Active Player, Non-Active Player	Priority ordering used by the stack and trigger resolution paths.
<i>CR</i>	Comprehensive Rules	Canonical MTG rules text used as implementation reference for mechanics.
$\mathcal{A}_{legal}$	Legal action set	Engine-generated action list; agents must choose exactly one member.
$\mathcal{D}$	Replay buffer	Collected trajectories (s_t, a_t, r_t) for model training and analysis.

3 The lineup

4 Glossary and symbols

5 Per-agent reference

5.1 RandomAgent

File: `src/agents/random_agent.py`. **Purpose:** baseline / smoke test.

```

1 def decide_action(self, game_state, legal_actions):
2     return self._rng.choice(legal_actions)

```

Configuration: `seed: int` (forwarded to `random.Random`). Failure modes: none. Deterministic given a seed.

5.2 HeuristicAgent

File: `src/agents/heuristic_agent.py`.

Decision flow.

- 1: `scored` $\leftarrow$ `[(priority(a), -tiebreak(a), a) for a in legal_actions]`
- 2: `scored.sort(descending)`
- 3: **return** `scored[0].a`

Priority table (`ACTION_PRIORITIES`):

Action type	Priority
PLAY_LAND	100
CAST_SPELL	90
DECLARE_ATTACKERS	80
DECLARE_BLOCKERS	70
ACTIVATE_ABILITY	60
PASS_PRIORITY	1
CONCEDE	0

Tiebreaks: prefer cheaper spells, prefer creatures with higher power when attacking. Failure modes: never holds up mana for instants (known limitation).

5.3 KGHeuristicAgent

File: `src/agents/kg_heuristic_agent.py`.

Wraps `HeuristicAgent` but rescores `CAST_SPELL` actions using the Neo4j graph through `src/knowledge/kg_query.py:score_card_for_state`:

$$\text{kg_score}(c) = 10 \cdot \mathbf{1}[c \in \text{NearComboMissingPieces}(s)] + \sum_{p \in \text{permanents}(s)} \text{strength}(\text{KG-edge}(c, p)). \quad (1)$$

The combined score is $\text{prio}(a) + \lambda \cdot \text{kg_score}(c)$ with $\lambda=5$. Configuration: `neo4j_uri`, `auth`, `enable_combo_lookup` (bool). Failure mode: if Neo4j is unreachable, falls back silently to plain `HeuristicAgent`.

5.4 OllamaAgent (and LLMAgent alias)

File: `src/agents/ollama_agent.py`.

Builds a textual board state via `format_state_for_llm()`, sends a chat completion request to `http://localhost:11434/api/chat` (model `gemma4:e2b`, temperature 0.2, `keep_alive=30m`, timeout 120s), and parses an integer action index out of the reply.

```
1 prompt = (  
2     "You are an MTG expert. Choose the BEST action by index.\n"  
3     f"State:\n{format_state_for_llm(state)}\n"  
4     f"Legal actions:\n{enumerate_actions(legal_actions)}\n"  
5     "Respond with ONLY the index."  
6 )  
7 resp = requests.post(OLLAMA_URL, json={"model": MODEL, "messages": [...]})  
8 idx = int(re.search(r"\d+", resp.json()["message"]["content"]).group())  
9 return legal_actions[idx % len(legal_actions)]
```

Failure modes: timeout $\rightarrow$ fall back to `HeuristicAgent`; non-integer reply $\rightarrow$ fall back to `RandomAgent` over a small legal-action subset.

5.5 ActiveInferenceAgent / KLControlActionPicker

Files: `src/agents/kl_control.py` (objective and planners), `src/agents/mtg_transition.py` (generative model), `src/integrations/phase_rs/kl_control_picker.py` (phase-rs driver).

Objective. Belief-space KL control — the canonical discretisation of continuous-time Active Inference [2]:

$$G(\pi) = \underbrace{\text{KL}(Q(\eta | \pi) \| P_0(\eta))}_{\text{control cost}} + \underbrace{\mathbb{E}_{Q(\eta|\pi)}[V(\eta)]}_{\text{risk}}, \quad V(\eta) = -\log p(\eta). \quad (2)$$

Note what is *not* there: no additive information-gain bonus and no ambiguity (likelihood-entropy) penalty. Both are surrogate terms with known failure modes, and neither is needed — exploration comes from evaluating π closed-loop over belief states, which is Fel'dbaum's dual control effect.

Components.

- **LatentState** — the plant coordinates η : life totals, board power, permanent counts, hand and library size, turn, and the two terminal flags. Deliberately small and *physical*.
- **PreferencePotential** — $V(\eta)$, the single place goals are encoded. Defined over game state, never over observations, which is what rules out transducer wireheading.
- **BeliefState** — categorical over hidden-information hypotheses, with `update()` (Bayes), `entropy()` (diagnostic only) and `expectation()`.
- **MTGTransitionModel** — analytic $p(\eta' | \eta, a, h)$. The hypothesis enters exactly where hidden information bites: a counterspell blanks a cast, removal blanks a board commitment, a hidden threat accelerates the opponent's clock.
- **BeliefSpaceKLControl** — the planner, with two engines.
- **LegacyEFE** — the two extant factorisations, for ablation only.

Planner engines.

Engine	Behaviour
<code>tree</code>	Exact closed-loop dynamic programming over the belief tree with observation branching. Reference semantics; exponential in horizon.
<code>mppi</code>	Model Predictive Path Integral. Samples K continuations from the passive prior, rolls each forward H steps, aggregates terminal potentials with the softmin operator. Default; linear in $K \cdot H$.

The passive prior is load-bearing. $q(a)$ is the reference measure the control cost is charged against, so $C_{\text{action}}(a) = -\log q(a) + c_{\text{move}}$. Heuristic, KG and LLM signals *shape* q via `mtg_transition.passive_prior(actions, extra_scores)`. This is where the four hand-set fusion weights went: they are no longer a separate additive stage with free coefficients.

Configuration (`KLControlConfig`).

Parameter	Default	Role
<code>horizon</code>	3	Planning depth H .
<code>beta</code>	4.0	Bounded-rationality precision. $\beta \rightarrow \infty$ gives the deterministic Bellman minimiser; $\beta \rightarrow 0$ gives the passive prior.
<code>rollouts</code>	48	K , MPPI sample count.
<code>engine</code>	<code>mppi</code>	<code>mppi</code> or <code>tree</code> .
<code>move_cost</code>	0.1	Additive stage cost, in nats.
<code>discount</code>	1.0	Per-step discount on the potential.
<code>seed</code>	—	Threads through both RNGs; required for determinism.

Objective selection for ablation. The picker takes `--objectivekl|efe_infogain|efe_ambiguity`. All three share the belief, transition model, potential, prior and seed, so they differ in nothing but the scoring function. Expected failure modes of the two legacy arms, in MTG terms:

- `efe_infogain` — noisy-TV capture: compulsive scry/surveil/cantrip loops that buy information with no bearing on the win condition.
- `efe_ambiguity` — scotophobia: refusing profitable attacks into possible blockers, or casts into open mana, purely because the outcome distribution is wide.

Invariant under test. `tests/test_kl_control.py` asserts that sharpening the belief without changing the plant state leaves the control cost unmoved. That is the structural property distinguishing (2) from `efe_infogain`, and it is the one a future refactor is most likely to break.

Decision traces. `PlanResult.as_trace()` emits the full decomposition — G , control cost, risk, the Gibbs policy, belief entropy, and the top- k candidates — keyed to the ontology’s `mtgd:` decision vocabulary, so a decision can be written into the knowledge graph as a `prov:Activity` rather than only into a log file.

5.6 WorldModelAgent

File: `src/agents/world_model_agent.py`. **Backbone:** `src/world_model/world_model.py`.

Training now uses the external `stable-pretraining` orchestration layer and the surrounding `stable-worldmodel` research stack rather than a bespoke in-repo trainer. Concretely, `scripts/train_stable_worldmodel.py` wraps the local MTG `WorldModel` in a `stable_pretraining.Module` and drives optimisation through `stable_pretraining.Manager`; this mirrors the software split used by `LeWorldModel`, which keeps the architecture and loss local while delegating training/runtime plumbing to the upstream libraries [1, 3, 4].

Components (V+M+C, after Ha & Schmidhuber).

$$\begin{aligned}
z_t &= \text{Enc}_V(o_t) && (\text{perceptual encoder, 768-d}) \\
g_t &= \text{Enc}_{KG}(\mathcal{G}, s_t) && (\text{GraphSAGE, 128-d}) \\
(h_t, c_t) &= \text{LSTM}_M([z_t; g_t], (h_{t-1}, c_{t-1})) \\
\hat{r}_t &= V(h_t) && (\text{value head}) \\
\pi(a|h_t) &= \text{softmax}(C(h_t)) && (\text{controller})
\end{aligned}$$

Decision rule. For each candidate action $a \in \mathcal{A}$, roll out K trajectories of horizon H in latent space (no actual rules-engine simulation):

$$\text{score}(a) = \frac{1}{K} \sum_{k=1}^K V(h_{t+H}^{(k)} \mid a_t = a). \quad (3)$$

Defaults: $K = 8$, $H = 10$, sampling temperature 1.15.

Checkpoint loading. `WorldModel.load` now tolerates three checkpoint shapes (canonical, JEPA-only, and free-form), warning-and-continuing rather than raising. The agent factory (`src/agents/__init__.py:_make_world_model`) falls back to a default-initialised network if the checkpoint file does not exist, so the harness can be smoke-tested before training.

Checkpoint troubleshooting.

- Shape mismatch in controller head: occurs when $|\mathcal{A}_{legal}|$ conventions differ between checkpoint and runtime. Re-export checkpoint after rebuilding action encoding.
- Missing JEPA keys: expected when loading pre-JEPA checkpoints; loader falls back to random init for missing tensors and logs a warning.
- KG context mismatch: if graph feature width changed, pass a matching checkpoint or train one epoch with `-resume` to adapt projection layers.

5.7 LLMFusionAgent

File: `src/agents/llm_fusion_agent.py`.

Combines four signals into a single ranking:

$$\text{score}(a) = \alpha_h s_{\text{heur}}(a) + \alpha_g s_{\text{KG}}(a) + \alpha_w s_{\text{WM}}(a) + \alpha_\ell \log p_{\text{LLM}}(a), \quad (4)$$

with default weights $(\alpha_h, \alpha_g, \alpha_w, \alpha_\ell) = (0.15, 0.15, 0.40, 0.30)$ from `FusionConfig`. When any signal exceeds the `obvious_threshold` (0.8), it short-circuits the vote to save an LLM call.

Algorithm 1 LLM-Fusion turn (simplified)

Require: state s_t , legal actions $\mathcal{A}$, weights α

- 1: $S_h \leftarrow \text{HeuristicAgent.score_all}(s_t, \mathcal{A})$
 - 2: $S_g \leftarrow \text{KG.score_all}(s_t, \mathcal{A})$
 - 3: **if** $\max_a S_h(a) > \tau_{\text{obvious}}$ **then return** $\arg \max_a S_h(a)$
 - 4: **end if**
 - 5: $S_w \leftarrow \text{WM.value_rollout}(s_t, \mathcal{A})$
 - 6: $\mathcal{A}^* \leftarrow \text{top-}N \text{ actions by } S_w$
 - 7: $S_\ell(a) \leftarrow \log p_{\text{LLM}}(a \mid \text{prompt}(s_t, \mathcal{A}^*))$
 - 8: **return** $\arg \max_{a \in \mathcal{A}^*} \alpha_h S_h(a) + \alpha_g S_g(a) + \alpha_w S_w(a) + \alpha_\ell S_\ell(a)$
-

FusionConfig defaults.

Parameter	Default	Role
<code>heuristic_weight</code>	0.15	Rules-aware tactical prior
<code>kg_weight</code>	0.15	Combo and synergy prior from Neo4j
<code>wm_weight</code>	0.40	Latent rollout value estimate
<code>llm_weight</code>	0.30	Language prior over legal indices
<code>top_n</code>	5	Candidate set handed to LLM stage
<code>obvious_threshold</code>	0.8	Short-circuit threshold for cheap decisions

One-turn message flow (ASCII).

```

1 priority_loop -> LLMFusionAgent.decide_action(state, legal_actions)
2   LLMFusionAgent -> HeuristicAgent.score_all: S_h
3   LLMFusionAgent -> KG query layer: S_g
4   alt max(S_h) > obvious_threshold
5     LLMFusionAgent --> priority_loop: argmax(S_h)
```

```

6     else
7         LLMFusionAgent -> WorldModelAgent.rollout_values: S_w
8         LLMFusionAgent -> OllamaAgent.log_probs(topN by S_w): S_l
9         LLMFusionAgent -> fuse(alpha_h*S_h + alpha_g*S_g + alpha_w*S_w + alpha_l*S_l)
10        LLMFusionAgent --> priority_loop: chosen legal action
11    end
12 priority_loop -> engine: apply chosen action

```

5.8 HumanAgent

File: `src/agents/human_agent.py`. Renders the legal actions to stdout and reads a 1-based index. Used by `examples/play_logged_game.py` for hand-debugging pathological states.

6 Debugging recipes

6.1 Agent loops on pass priority

Symptoms: repeated `PASS_PRIORITY` despite non-trivial board states.

Checklist:

- Inspect `legal_actions` logs from `src/orchestrator/priority_loop.py`; if only pass/concede exist, the issue is engine-side rather than policy-side.
- Verify stack resolution path in `src/engine/stack.py` and trigger enqueueing in `src/engine/triggered_abilities.py`.
- Confirm current phase allows the expected action class (main phase vs combat priority windows).

6.2 Agent never attacks

Symptoms: board develops, but attackers are never declared.

Checklist:

- Check `src/engine/combat.py:legal_attackers` output for summoning-sickness and tap-state filtering.
- Ensure power/toughness modifiers from equipment and temporary effects reach `effective_power` and `effective_toughness`.
- For heuristic agents, inspect `ACTION_PRIORITIES` and tiebreak functions for accidental over-penalisation of combat lines.

6.3 World-model agent regresses after checkpoint swap

Symptoms: sudden Elo drop or random-looking actions after loading a new checkpoint.

Checklist:

- Confirm checkpoint compatibility warnings from `WorldModel.load`; missing keys should be expected and logged.
- Run a deterministic smoke pod (seed fixed) and compare trace-level `wm` scores before/after swap.
- Recompute normalisation statistics if latent feature scales changed between training runs.

6.4 LLM fusion hangs or times out

Symptoms: long think times or fallback-only behaviour.

Checklist:

- Verify Ollama endpoint and model are available (see docs/OLLAMA_SETUP.md).
- Lower `top_n` and increase `obvious_threshold` to reduce LLM calls under load.
- Confirm timeout fallback path returns a legal action and records the fallback reason in reasoning traces.

7 Reasoning trace contract

Every agent except Random/Human emits a structured JSON record on each decision into `state.log_jsonl(...)`:

```
1 {
2   "turn": 7, "phase": "main1", "agent": "llm_fusion",
3   "chosen_action_idx": 3,
4   "scores": {"heuristic": [...], "kg": [...], "wm": [...], "llm": [...]},
5   "weights": {"heur": 0.15, "kg": 0.15, "wm": 0.40, "llm": 0.30},
6   "elapsed_ms": 412,
7   "explanation": "WM rollout favours T+8 board state with..."
8 }
```

Trace files land at `runs/<exp>/game_NNN.jsonl` alongside the human-readable `game_NNN.log`.

Minimal deterministic trace example (heuristic).

```
1 {
2   "turn": 3,
3   "phase": "main1",
4   "agent": "heuristic",
5   "chosen_action_idx": 1,
6   "scores": {"heuristic": [100.0, 90.0, 1.0]},
7   "weights": {"heur": 1.0},
8   "elapsed_ms": 1,
9   "explanation": "play land has highest static priority"
10 }
```

Fusion trace example (all channels active).

```
1 {
2   "turn": 7,
3   "phase": "main1",
4   "agent": "llm_fusion",
5   "chosen_action_idx": 3,
6   "scores": {
7     "heuristic": [0.32, 0.81, 0.11, 0.44],
8     "kg": [0.05, 0.10, 0.62, 0.18],
9     "wm": [0.40, 0.21, 0.73, 0.68],
10    "llm": [-1.50, -0.20, -1.10, -0.35]
11  },
12   "weights": {"heur": 0.15, "kg": 0.15, "wm": 0.40, "llm": 0.30},
13   "elapsed_ms": 412,
14   "explanation": "wm+llm agree on index 3 after top-N pruning"
15 }
```

Agent	Test file
RandomAgent	tests/test_random_agent.py
HeuristicAgent	tests/test_heuristic_agent.py
KGHeuristicAgent	tests/test_kg_heuristic_agent.py
OllamaAgent (mocked)	tests/test_ollama_agent.py
ActiveInferenceAgent	tests/test_active_inference_agent.py
WorldModelAgent	tests/test_world_model_agent.py
LLMFusionAgent	tests/test_llm_fusion_agent.py

8 Test pointers

Smoke commands.

```

1 .\venv\Scripts\python.exe -m pytest tests/ -q
2 .\venv\Scripts\python.exe examples\play_edh_pod.py --max-turns 6 --seed 7 --model none
3 scripts\run_overnight_benchmarks.bat --quick

```

9 Theoretical foundations (extended)

9.1 POMDPs and partial observability

A finite-horizon POMDP is the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma, b_0)$. The optimal value function over beliefs satisfies

$$V^*(b) = \max_a \left[\sum_s b(s) R(s, a) + \gamma \sum_o \Pr(o | b, a) V^*(b'_o) \right], \quad (5)$$

where b'_o is the Bayes update of b after action a and observation o . Exact solution is intractable for $|\mathcal{S}| > 10^{800}$; all our policies are approximations.

9.2 World models as latent imagination

Following Ha & Schmidhuber and Hafner et al. (Dreamer), we factor the agent as V (encoder), M (recurrent dynamics), and C (controller). The one-step JEPA stage jointly optimises the observation encoder and predictor, using the same encoder under stop-gradient on the target branch:

$$\mathcal{L}_{\text{JEPA}} = \mathbb{E}_{\mathcal{D}} [\text{MSE}(\hat{z}_{t+1}, \text{sg}(E_{\phi}(o_{t+1}))) + \beta_J \text{KL}(q_{\phi}(z|o_t) \parallel \mathcal{N}(0, I))], \quad (6)$$

where $\mathcal{D}$ contains consecutive transitions and optional KG context is suppressed in the notation. Prediction targets an embedding, not a complete symbolic successor state. β_J (**-jepa-beta**, default 1.0) weights encoder regularisation; the reference is a standard normal, not an LSTM-conditioned prior. Dream search currently uses separate recurrent dynamics and accumulates predicted rewards. This stage does not train that recurrent model or establish an H -step JEPA rollout objective. In the current repository this loss is still computed by the local MTG model, but the optimisation loop, callback stack, and checkpoint orchestration are delegated to **stable-pretraining**; **stable-worldmodel** remains the upstream runtime substrate around data collection and world-model evaluation conventions [1, 3].

9.3 Free energy, active inference, and optimal control

Friston’s free-energy principle frames perception and action as joint inference. The variational free energy is

$$F[q] = \underbrace{\mathbb{E}_q[\log q(s) - \log p(o, s)]}_{\text{evidence bound}} = \text{KL}(q \parallel p(s|o)) - \log p(o). \quad (7)$$

Action selection minimises *expected* free energy. Kaufmann [2] establishes that on path space this functional is the relative entropy of the controlled path measure against a preference-tilted target, which by Girsanov’s theorem is finite-horizon path-integral stochastic MPC; its canonical discretisation is belief-space KL control,

$$G(\pi) = \text{KL}(Q(\eta \mid \pi) \parallel P_0(\eta)) + \mathbb{E}_{Q(\eta \mid \pi)}[V(\eta)]. \quad (8)$$

The conventional pragmatic/epistemic split is therefore not a derivation but a pair of surrogate objectives, which agree only when the sensory preference is exactly the pushforward of the state preference. Under a finite deliberation budget the hard min becomes a softmin and policy selection follows $p_\beta(\pi) \propto q(\pi)e^{-\beta G(\pi)}$; a single precision β interpolates between the deterministic Bellman minimiser and the passive prior. This is what `src/agents/kl_control.py` implements.

9.4 Knowledge-graph grounding

Cards form an inductive graph $\mathcal{G} = (V, E)$ with edges for *combo*, *synergy*, *counters*, and *enables* (see `src/knowledge/`). GraphSAGE produces inductive node embeddings $\mathbf{h}_v \in \mathbb{R}^{128}$ that are concatenated with the perceptual latent z_t before the recurrent dynamics. This is the V+M+C model with an additional *graph* channel Enc_{KG} .

9.5 LLM-as-policy under legal-action constraints

The LLM never freely generates text. Given the prompt $\rho(s_t, \mathcal{A})$, we read off

$$\log p_{\text{LLM}}(a \mid s_t) = \text{LM}(\text{idx}(a) \mid \rho(s_t, \mathcal{A})) \quad (9)$$

and mask any index outside $\{0, \dots, |\mathcal{A}| - 1\}$. This is the standard “constrained decoding” trick and prevents illegal hallucinated moves.

9.6 Self-play promotion as fixed-point iteration

The iterative training loop in `scripts/train_pipeline.py` implements a champion-vs-challenger schema: a challenger trained on the latest replay buffer plays a best-of- N against the current champion; promotion happens iff the win rate exceeds 0.55. This is a stochastic-approximation fixed-point on the policy improvement operator and converges (in expectation) provided the dynamics model is locally accurate.

10 Ontology-backed decision provenance

Reasoning traces answer “what did the agent score?”. They do not answer “why is this decision defensible?”, because a JSON blob in `runs/` has no schema a third party can rely on and no link to the domain vocabulary. The v2.0 ontology therefore carries a decision module in its own namespace, `mtgd:`, so decisions can be written into the same graph as the game they occur in.

Term	Aligned to	Carries
<code>mtgd:Decision</code>	<code>mtg:GameAction</code> , <code>prov:Activity</code>	one action selection
<code>mtgd:ExpectedFreeEnergy</code>	<code>dul:Parameter</code>	<code>controlCost</code> , <code>risk</code>
<code>mtgd:BeliefState</code>	<code>dul:InformationObject</code>	<code>beliefEntropy</code>
<code>mtgd:Policy</code>	<code>dul:Plan</code>	<code>precision</code> (β)
<code>mtgd:PreferencePotential</code>	<code>dul:Parameter</code>	the goal specification

Because `mtgd:Decision` is a `prov:Activity`, “why did seat 0 attack on turn 6?” is a graph query returning the objective decomposition, the belief it was taken under, and the policy

precision in force — rather than a log grep. This is also what makes the agent layer auditable in the same way the card data is: an explanation grounded in domain terms with provenance attached, instead of a saliency score.

The mapping is intentionally lossless in one direction only. Traces are the source of truth at runtime (cheap, always written); graph materialisation is a post-processing step over the trace files, so a Neo4j outage can never block play.

11 Where to look next

- Companion paper: `paper/opposition_agents_mtg.tex`.
- Single source of truth: `IMPLEMENTATION_PLAN.md`.
- Control objective: `src/agents/kl_control.py`, reviewed in `docs/ACTIVE_INFERENCE.md`.
- Ontology: `data/ontology/mtg-ontology-v2.0.ttl`, validated by `scripts/validate_ontology.py`.
- Competency questions: `data/competency_questions.yaml`.
- Engine docs: `ARCHITECTURE.md`, `DEVELOPMENT.md`.
- World-model design: `docs/WORLD_MODEL_DESIGN.md`.
- Opponent modelling: `docs/OPPONENT_MODELING_WITH_DECKLIST.md`.

12 References

- [1] Randall Balestriero, Hugues Van Assel, Sami BuGhanem, and Lucas Maes. `stable-pretraining-v1`: Foundation model research made simple. arXiv:2511.19484, 2025. URL <https://arxiv.org/abs/2511.19484>. Software: <https://github.com/galilai-group/stable-pretraining>.
- [2] Rafael Kaufmann. Active Inference is Optimal Control. SSRN preprint, 2026. URL <https://doi.org/10.2139/ssrn.7504418>.
- [3] Lucas Maes, Quentin Le Lidec, Dan Haramati, Nassim Massaudi, Damien Scieur, Yann LeCun, and Randall Balestriero. `stable-worldmodel-v1`: Reproducible world modeling research and evaluation. arXiv:2602.08968, 2026. URL <https://arxiv.org/abs/2602.08968>. Software: <https://github.com/galilai-group/stable-worldmodel>.
- [4] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. `LeWorldModel`: Stable end-to-end joint-embedding predictive architecture from pixels. arXiv:2603.19312, 2026. URL <https://arxiv.org/abs/2603.19312>. Software: <https://github.com/lucas-maes/le-wm>.
- [5] phase-rs contributors. `phase-rs`: An open-source Magic: The Gathering rules engine and game client. Rust software and project repository, 2026. URL <https://github.com/phase-rs/phase>. Accessed 5 October 2026; engine revision 1191bba65048c83fbbd5a56dfd5f419ee0ead67a, protocol v106.